



Autarquia Educacional do Vale do São Francisco
Faculdade de Petrolina
Curso de Ciência da Computação

AgroPredict — Documentação do Projeto

Francklin Leandro, Gabriel Martins, Pedro Henrique Campos, Victor Emanuel e Wagner Lucas

Professor: Mateus Amorim Silva

Petrolina

2026

SUMÁRIO

1. VISÃO GERAL	3
2. ESTRUTURA DO PROJETO	3
3. PIPELINE DE DADOS	4
4. ENGENHARIA DE FEATURES	5
5. MODELO DE MACHINE LEARNING	6
6. API REST	8
7. DASHBOARD INTERATIVO	9
8. MONITORAMENTO DE DRIFT	10
9. DEPENDÊNCIAS PRINCIPAIS	10
10. COMO EXECUTAR	11

1. VISÃO GERAL

O AgroPredict é um projeto de ciência de dados end-to-end voltado para o agronegócio do semiárido brasileiro. Ele coleta dados climáticos históricos reais de Petrolina (PE) via API pública, processa e transforma esses dados em features relevantes, treina um modelo de classificação binária e disponibiliza as previsões por meio de uma API REST e um dashboard interativo.

Problema: Irrigar desnecessariamente desperdiça água e recursos; não irrigar nos momentos certos compromete a produção. O modelo responde automaticamente à pergunta: "Hoje é necessário irrigar?"

Abordagem: Classificação supervisionada com regras agronômicas para geração do rótulo target (precisa_irrigar).

2. ESTRUTURA DO PROJETO

AgroPredict/

```

|
|— data/
|   |— raw/          # Dados brutos coletados da API
|   |— processed/    # Dados após limpeza e renomeação
|   |— final/        # Dataset final com features
|
|— models/
|   |— modelo_irrigacao.pkl # Modelo treinado serializado
|
|— notebooks/
|   |— 01_eda_inicial.ipynb # Análise exploratória dos dados
|
|— src/
|   |— data/
|       |— coletar_dados.py # Coleta de dados via Open-Meteo
|       |— preparar_dados.py # Limpeza e padronização

```

```

|   |— features/
|   |   └─ criar_features.py  # Engenharia de features e target
|   |— models/
|   |   └─ treinar_modelo.py  # Treinamento com MLflow
|   |   └─ avaliar_modelo.py # Avaliação do modelo
|   |— monitoring/
|   |   └─ detectar_drift.py  # Monitoramento de drift climático
|   └─ api/
|       └─ main.py            # API FastAPI
|
|— dashboard/
|   └─ app.py                 # Dashboard Gradio
|
└─ requirements.txt

```

3. PIPELINE DE DADOS

O pipeline segue o fluxo clássico de ciência de dados, com cada etapa encapsulada em um módulo dedicado.

1. Coleta de Dados — `src/data/coletar_dados.py`

Os dados são obtidos da Open-Meteo Archive API, uma API pública de dados meteorológicos históricos, sem necessidade de autenticação.

Localização: Petrolina, PE — latitude -9.38, longitude -40.50

Variáveis coletadas (escala diária):

Variável original (API)	Descrição
temperature_2m_max	Temperatura máxima diária (°C)
temperature_2m_min	Temperatura mínima diária (°C)

precipitation_sum	Precipitação acumulada diária (mm)
wind_speed_10m_max	Velocidade máxima do vento (km/h)
relative_humidity_2m_mean	Umidade relativa média (%)
shortwave_radiation_sum	Radiação solar acumulada (MJ/m ²)
et0_fao_evapotranspiration	Evapotranspiração de referência FAO

Período: 2025-01-01 a 2025-12-31

Saída: data/raw/dados_climaticos_petrolina.csv

2. Preparação dos Dados — src/data/preparar_dados.py

Etapa de limpeza e padronização. As operações realizadas são:

- Renomeação das colunas para nomes em português (mais legíveis e consistentes com o domínio)
- Conversão da coluna data para o tipo datetime
- Ordenação cronológica dos registros
- Remoção de duplicatas

Saída: data/processed/petrolina_clima_tratado.csv

4. ENGENHARIA DE FEATURES

Módulo: src/features/criar_features.py

Sobre o dataset processado, são criadas novas variáveis derivadas e o rótulo target:

Features derivadas

Feature	Cálculo	Justificativa agrônômica
mes	Mês extraído da data	Captura sazonalidade climática

amplitude_termica	$\text{temperatura_maxima} - \text{temperatura_minima}$	Estresse hídrico é maior em amplitudes elevadas
media_temperatura	$(\text{temperatura_maxima} + \text{temperatura_minima}) / 2$	Indicador geral de calor do dia
indice_seca	$(\text{temperatura_maxima} \times \text{evapotranspiracao}) / (\text{umidade_media} + 1)$	Índice composto de demanda hídrica vs. disponibilidade

Target: precisa_irrigar

A variável alvo é gerada por regras agronômicas adaptadas ao semiárido:

python

```
df["precisa_irrigar"] = (
    (df["precipitacao"] < 2)
    & (df["temperatura_maxima"] > 32)
    & (df["umidade_media"] < 55)
    & (df["evapotranspiracao"] > 6)
).astype(int)
```

O campo é codificado como 1 (irrigar) ou 0 (não irrigar).

Saída: data/final/petrolina_dataset_final.csv

5. MODELO DE MACHINE LEARNING

Módulo: src/models/treinar_modelo.py

Algoritmo

Random Forest Classifier (scikit-learn)

Hiperparâmetro	Valor
n_estimators	100
random_state	42

Features de entrada (11 variáveis)

```
features = [  
    "temperatura_maxima",  
    "temperatura_minima",  
    "precipitacao",  
    "vento_maximo",  
    "umidade_media",  
    "radiacao_solar",  
    "evapotranspiracao",  
    "mes",  
    "amplitude_termica",  
    "media_temperatura",  
    "indice_seca",  
]
```

Divisão treino/teste

- **Treino:** 80% dos dados
- **Teste:** 20% dos dados
- **Estratificação:** sim (stratify=y), para preservar a proporção das classes

Rastreamento de Experimentos — MLflow

Cada execução de treino registra automaticamente no MLflow:

- **Parâmetros:** nome do modelo, número de árvores
- **Métricas:** acurácia no conjunto de teste
- **Artefato:** o próprio modelo serializado

O modelo treinado é salvo localmente em models/modelo_irrigacao.pkl via joblib.

Avaliação — src/models/avaliar_modelo.py

A avaliação completa inclui:

- **Matriz de confusão** (verdadeiros/falsos positivos e negativos)
- **Relatório de classificação** com precisão, recall, F1-score por classe

6. API REST

Módulo: src/api/main.py — Framework: **FastAPI**

Endpoints

GET /

Verifica se a API está ativa.

Resposta:

json

```
{ "mensagem": "AgroPredict API funcionando" }
```

POST /predict

Recebe dados climáticos de um dia e retorna a previsão de irrigação.

Body (JSON):

json

```
{  
  "temperatura_maxima": 38.5,  
  "temperatura_minima": 24.0,  
  "precipitacao": 0.0,  
  "vento_maximo": 18.3,  
  "mes": 7,  
  "amplitude_termica": 14.5,  
  "media_temperatura": 31.25,  
  "umidade_media": 45.0,  
  "radiacao_solar": 22.1,  
  "evapotranspiracao": 7.3,  
  "indice_seca": 4.2  
}
```

Resposta:

json

```
{ "precisa_irrigar": true }
```


7. DASHBOARD INTERATIVO

Módulo: dashboard/app.py — Framework: **Gradio**

O dashboard possui quatro abas:

Aba 1 — Resumo dos Dados

Exibe um resumo estatístico do período selecionado (mês ou ano inteiro):

- Total de dias analisados
- Dias com necessidade de irrigação
- Temperatura máxima média
- Chuva acumulada
- Umidade média
- Evapotranspiração média

Aba 2 — Gráficos Climáticos

Visualização interativa (linha ou colunas) das variáveis climáticas ao longo do tempo, filtrável por mês e variável.

Variáveis disponíveis: temperatura máxima/mínima, precipitação, vento, umidade, radiação solar, evapotranspiração, índice de seca, necessidade de irrigação.

Aba 3 — Previsão de Irrigação

O usuário informa uma data (no formato YYYY-MM-DD) e o sistema:

1. Busca os dados daquele dia na base
2. Executa o modelo treinado
3. Exibe se há ou não necessidade de irrigação, junto com todos os valores climáticos do dia

Aba 4 — MLflow

Carrega e exibe os experimentos de treinamento registrados no MLflow, incluindo ID da execução, status, acurácia, nome do modelo e data do treino.

8. MONITORAMENTO DE DRIFT

Módulo: src/monitoring/detectar_drift.py

Implementa um monitoramento simples de **data drift climático** comparando dois semestres:

- **Base de referência:** Janeiro a Junho
- **Base atual:** Julho a Dezembro

Para cada variável monitorada, calcula a diferença percentual entre as médias:

$$\text{diferença (\%)} = ((\text{média_atual} - \text{média_referência}) / \text{média_referência}) \times 100$$

Alerta: Se a diferença for superior a **20%**, o status é marcado como DRIFT DETECTADO.

Variáveis monitoradas: temperatura máxima, precipitação, umidade média, radiação solar, evapotranspiração, índice de seca.

9. DEPENDÊNCIAS PRINCIPAIS

Biblioteca	Versão	Uso
pandas	2.3.3	Manipulação de dados
scikit-learn	1.8.0	Modelo de ML e métricas
mlflow	3.12.0	Rastreamento de experimentos
fastapi	0.136.1	API REST
gradio	6.14.0	Dashboard interativo
plotly	6.7.0	Gráficos interativos
joblib	—	Serialização do modelo
requests	2.34.0	Coleta de dados via HTTP
numpy	2.2.3	Operações numéricas

10. COMO EXECUTAR

1. Instalar dependências

```
pip install -r requirements.txt
```

2. Executar o pipeline completo

```
# Coleta de dados
```

```
python src/data/coletar_dados.py
```

```
# Preparação
```

```
python src/data/preparar_dados.py
```

```
# Criação de features
```

```
python src/features/criar_features.py
```

```
# Treinamento
```

```
python src/models/treinar_modelo.py
```

```
# Avaliação
```

```
python src/models/avaliar_modelo.py
```

3. Monitorar drift

```
python src/monitoring/detectar_drift.py
```

4. Iniciar a API

```
uvicorn src.api.main:app --reload
```

Acesse a documentação automática em: <http://localhost:8000/docs>

5. Iniciar o Dashboard

```
python dashboard/app.py
```

O Gradio abrirá automaticamente no navegador.